

实验报告

系统开发工具基础 第 4 周

姓 名： 王本熙

学 号： 25020007117

日 期： 2026 年 9 月 14 日

平 台： Windows + WSL (Ubuntu)

目录

1 实验目的	2
2 实验环境	2
3 q13: 建立可执行的本地质量门禁	2
3.1 题目要求	2
3.2 实验步骤	3
3.3 运行结果与截图	4
3.4 结果分析	4
4 q14: 让 Make 只重建真正受影响的产物	6
4.1 题目要求	6
4.2 实验步骤	6
4.3 运行结果与截图	7
4.4 结果分析	7
5 q15: 把本地 API 数据转换为可读报告	7
5.1 题目要求	7
5.2 实验步骤	7
5.3 运行结果与截图	9
5.4 结果分析	9
6 q16: 修复并交付一个陌生的小型工具仓库	11
6.1 题目要求	11
6.2 实验步骤	11
6.3 运行结果与截图	12
6.4 结果分析	12
7 代码与仓库	12
7.1 Git 提交记录	14
8 结论	14

第 1 章 实验目的

本实验旨在掌握以下内容：

- 建立可执行的本地质量门禁：`ruff format --check`、`ruff check`、`pytest` 串联为 `check.sh`，验证返回 0（q13）；
- 编写 Makefile 实现增量构建：准确声明数据文件、脚本和中间产物的依赖关系，`all` 与 `clean` 声明为 `.PHONY`（q14）；
- 使用本地 HTTP 服务 + `curl` + `jq` 筛选 JSON 数据并生成 Markdown 报告（q15）；
- 综合测试：临时改坏实现确认测试失败，再定位修复，编写含 `check`、`build`、`clean` 的 Makefile，计算 wheel 的 SHA-256 并提交为单一 Git 提交（q16）。

第 2 章 实验环境

表 1: 实验环境

项目	配置
操作系统	Windows + WSL (Ubuntu 24.04)
Shell	bash
Python	3.12.3
编辑器	VS Code / Trae IDE / micro
版本管理	Git
远程仓库	GitHub
构建工具	Make / <code>python -m build</code>
静态检查	<code>ruff</code> (format + check)
测试	<code>pytest</code>
JSON 处理	<code>jq</code> 1.7.1

第 3 章 q13: 建立可执行的本地质量门禁

3.1 题目要求

主题：代码质量。复用第 10 题完成的 `greetlab` 包，在 q13 中建立一个轻量本地质量检查：

- 复制 q10 为 q13；在 `pyproject.toml` 中加入 `ruff` 和 `pytest` 的最小配置；
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言；
- 运行 `ruff format`、`ruff check` 和 `pytest`，修复全部问题，不得全局忽略规则；

- 编写 `check.sh`, 依次执行 `ruff format --check`、`ruff check` 和 `pytest`; 运行并保证返回 0。

3.2 实验步骤

复制 q10 为 q13, 更新 `pyproject.toml` 加入 `ruff` 与 `pytest` 配置:

```
1 [tool.ruff]
2 line-length = 88
3 target-version = "py312"
4
5 [tool.ruff.lint]
6 select = ["E", "F", "I", "W"]
7
8 [tool.ruff.format]
9 quote-style = "double"
10
11 [tool.pytest.ini_options]
12 testpaths = ["tests"]
```

两个测试:

```
1 # tests/test_cli.py
2 import sys
3 import pytest
4 from greetlab.cli import main
5
6 def test_normal_name(monkeypatch, capsys):
7     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "World"])
8     main()
9     captured = capsys.readouterr()
10    assert captured.out.strip() == "Hello, World!"
11
12 def test_blank_name_raises_system_exit(monkeypatch, capsys):
13     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
14     with pytest.raises(SystemExit) as excinfo:
15         main()
16     assert excinfo.value.code == 2
17     captured = capsys.readouterr()
18    assert captured.out == ""
```

`check.sh`:

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 echo "=== ruff format check ==="
5 ruff format --check .
```

```
6
7 echo "=== ruff check ==="
8 ruff check .
9
10 echo "=== pytest ==="
11 pytest
12
13 echo "=== all checks passed ==="
```

运行验证：

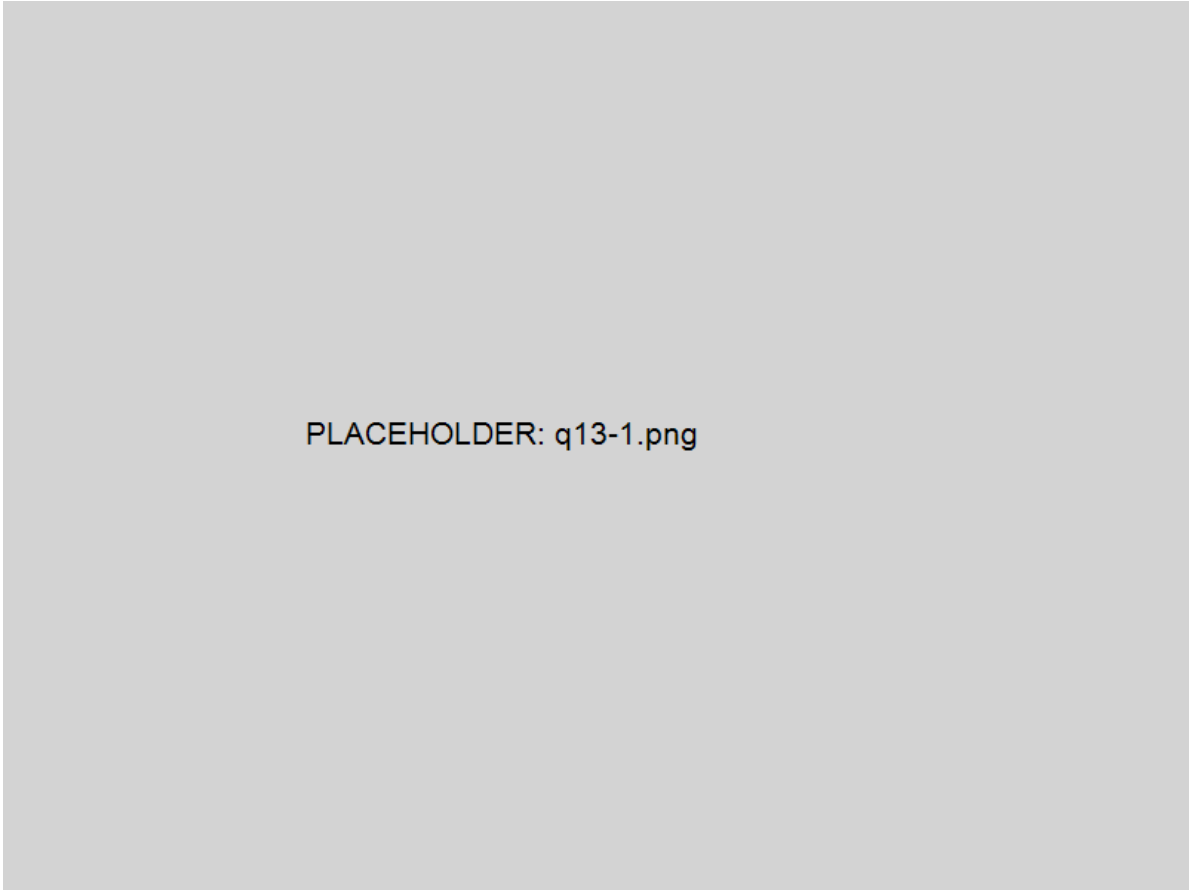
```
1 cd q13
2 python3 -m venv venv && source venv/bin/activate
3 pip install -e . ruff pytest
4 bash check.sh
5 # === ruff format check ===
6 # 3 files already formatted
7 # === ruff check ===
8 # All checks passed!
9 # === pytest ===
10 # collected 2 items
11 # tests/test_cli.py .. [100%]
12 # 2 passed in 0.01s
13 # === all checks passed ===
```

3.3 运行结果与截图

图 1 展示了 pytest 测试通过、环境配置与 check.sh 完整运行。

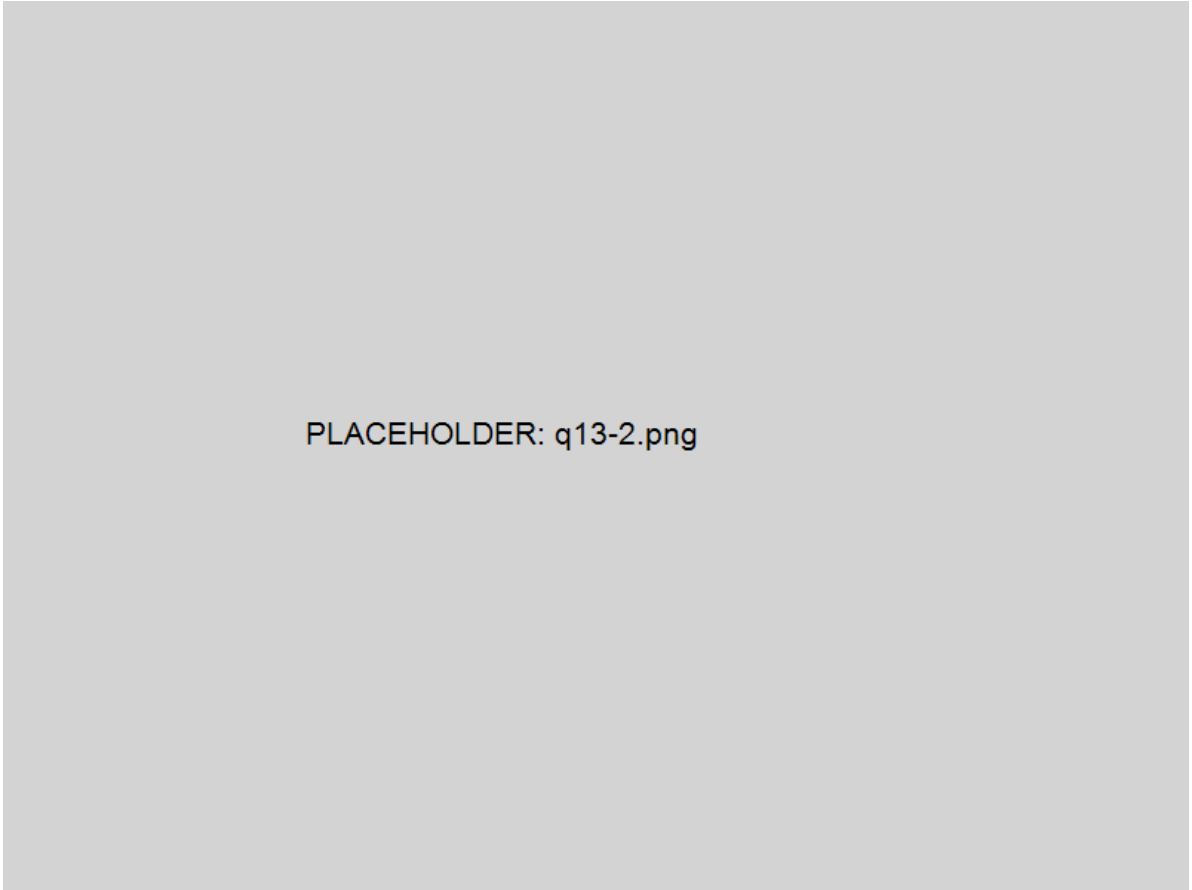
3.4 结果分析

- **pyproject.toml 配置:** [tool.ruff] 指定 line-length = 88 (PEP 8 默认) 和 target-version = "py312"; [tool.ruff.lint] select = ["E", "F", "I", "W"] 启用 pycodestyle 错误、pyflakes、isort、pycodestyle 警告; [tool.pytest.ini_options] testpaths = ["tests"] 让 pytest 自动发现测试;
- **两个测试:** test_normal_name 验证正常输入输出 Hello, World!; test_blank_name 验证空白姓名抛 SystemExit(2)。两者都使用 monkeypatch 注入 sys.argv、capsys 捕获 stdout, 隔离且无需真实命令行;
- **check.sh 串联:** set -euo pipefail 保证任何环节失败即退出, 不会出现“ruff check 失败但 pytest 仍跑”的情况; ruff format --check (仅检查不改写)、ruff check (静态分析)、pytest (行为验证) 三层防护, 返回值 0 即全部通过;
- **src-layout 陷阱:** 运行 pytest 前必须 pip install -e . 以可编辑模式安装包, 否则会报 ModuleNotFoundError: No module named 'greetlab'。



PLACEHOLDER: q13-1.png

(a) `pytest: collected 2 items, 2 passed in 0.01s`



PLACEHOLDER: q13-2.png

(b) 创建 `venv`、编辑 `cli.py` 与 `pyproject.toml`、`pip` 配置清华源

第 4 章 q14: 让 Make 只重建真正受影响的产物

4.1 题目要求

主题：元编程——构建系统。在 q14 中按给定内容创建三个源文件，只需完成 Makefile：

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md；
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY；
- 验证首次 make 生成两个产物；第二次无改动时不执行配方；touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

4.2 实验步骤

按题目给定内容创建四个源文件：

```
1 # data.csv
2 name,value
3 alpha,2
4 beta,3
5 gamma,5
6 # stats.py / build_report.py / report.md 按题目给定
```

Makefile:

```
1 .PHONY: all clean
2
3 all: report.txt
4
5 stats.txt: data.csv stats.py
6     python3 stats.py
7
8 report.txt: stats.txt build_report.py report.md
9     python3 build_report.py
10
11 clean:
12     rm -f stats.txt report.txt
```

增量构建验证：

```
1 cd q14
2 make          # 首次: python3 stats.py → python3 build_report.py
3 make          # 二次: Nothing to be done for 'all'
4 touch data.csv
5 make          # 三次: data.csv 时间戳变了，两个产物都重建
6 make clean    # rm -f stats.txt report.txt
```

4.3 运行结果与截图

图 2 展示了增量构建的完整验证。

4.4 结果分析

- **依赖链正确:** `all` $\rightarrow$ `report.txt` $\rightarrow$ `stats.txt` $\rightarrow$ `data.csv` + `stats.py`, `report.txt` 额外依赖 `build_report.py` 和 `report.md`; Make 通过比较目标文件与依赖文件的修改时间戳决定是否重建;
- **.PHONY 声明:** `.PHONY: all clean` 告诉 Make `all` 和 `clean` 是命令名而非目标文件名, 即使当前目录下存在同名文件也会执行对应配方;
- **增量验证:** 首次 `make` 依次运行 `python3 stats.py` 和 `python3 build_report.py`; 二次 `make` 无任何文件改动, 输出 `Nothing to be done for 'all'`; `touch data.csv` 只更新 `data.csv` 的时间戳, Make 检测到 `stats.txt` 比依赖旧, 重建 `stats.txt`, 进而 `report.txt` 的依赖也变新, 联动重建;
- **中间产物清理:** `clean: rm -f stats.txt report.txt` 只删除目标文件, 保留源码, 符合 `make clean` 的通用约定。

第 5 章 q15: 把本地 API 数据转换为可读报告

5.1 题目要求

主题: 大杂烩——API、jq、CLI 约定与 Markdown。在 q15 中创建给定 JSON 和本地 HTTP 服务, 再用 curl 与 jq 生成简短 Markdown 报告:

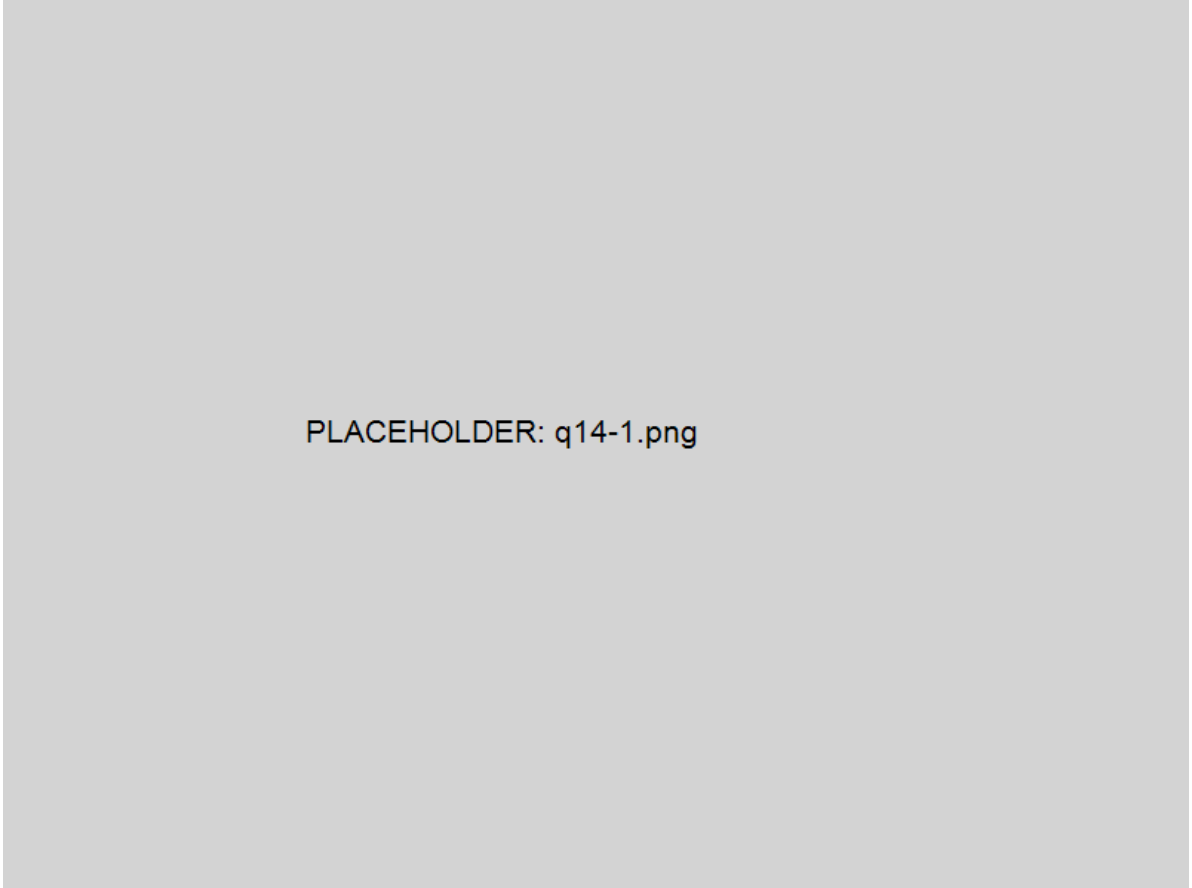
- 把给定 JSON 保存为 `packages.json`, 在 q15 目录运行 `python -m http.server 8000`;
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`;
- 使用 jq 筛选 `status` 为 `active` 且 `downloads` 不少于 100 的记录, 并按 `downloads` 降序、`name` 升序排列;
- 编写 `api_report.sh`, 把结果生成 `summary.md`, 包含标题和 `name/version/downloads` 三列表格。

5.2 实验步骤

`packages.json` 按题目给定内容保存。HTTP 服务后台启动:

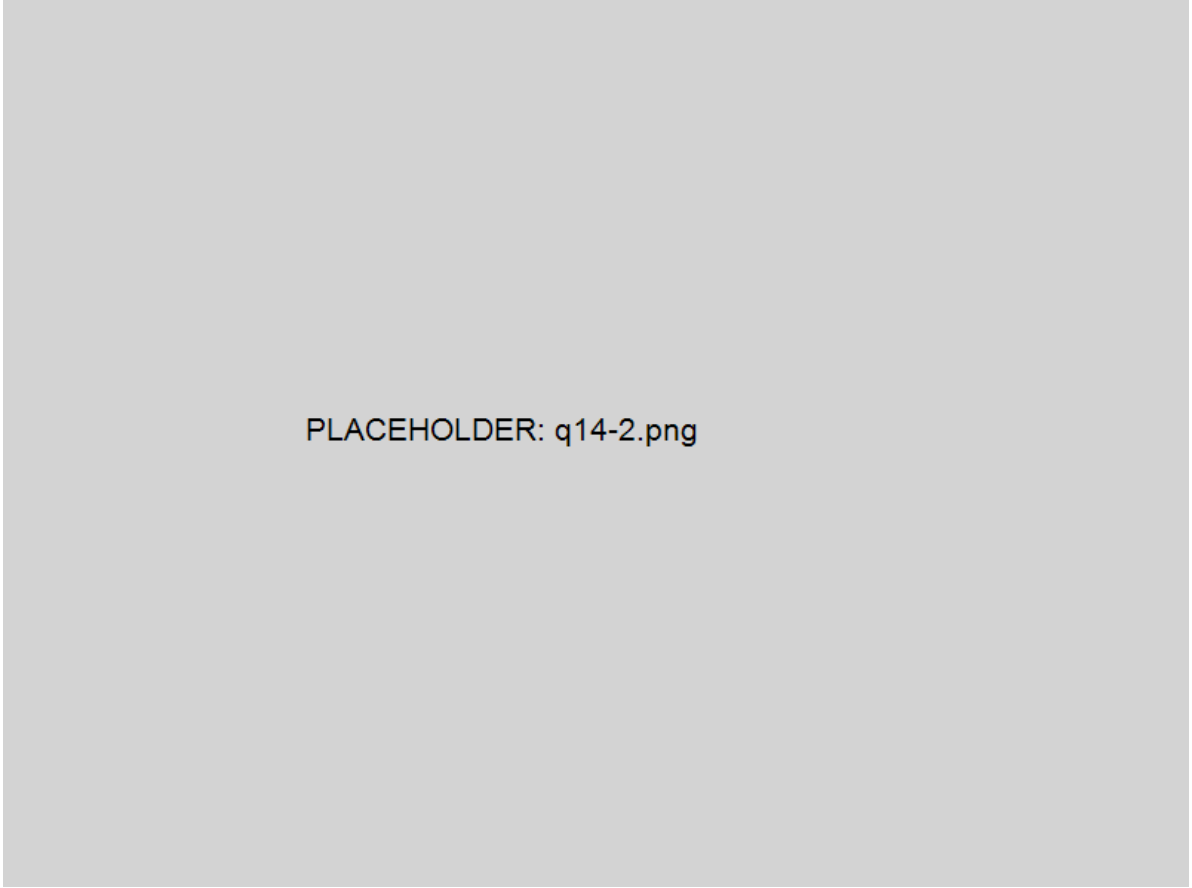
```
1 cd q15
2 python3 -m http.server 8000 &      # 后台启动 HTTP 服务
3 sudo apt install -y jq             # 安装 jq (WSL 默认未装)
```

`api_report.sh`:



PLACEHOLDER: q14-1.png

(a) 创建 `data.csv`、`stats.py`、`build_report.py`、`report.md`、`Makefile`



PLACEHOLDER: q14-2.png

(b) 首次 `make` 生成两产物 → 二次 `make`: `Nothing to be done` → `touch data.csv` 后重建 → `make clean` 删除生成物

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  curl -fsS http://127.0.0.1:8000/packages.json \
5  | jq -s '[] | select(.status == "active" and .downloads >= 100)] | sort_by(-.
      downloads, .name)' \
6  | jq -r '
7      "# Active Packages (downloads 100)",
8      "",
9      "| Name | Version | Downloads |",
10     "|-----|-----|-----|",
11     (.[] | " | \(.name) | \(.version) | \(.downloads) |")
12     ' > summary.md
13
14  cat summary.md

```

运行：

```

1  chmod +x api_report.sh
2  ./api_report.sh
3  # [14/Sep/2026 16:32:58] "GET /packages.json HTTP/1.1" 200 -
4  cat summary.md
5  # # Active Packages (downloads 100)
6  #
7  # | Name | Version | Downloads |
8  # |-----|-----|-----|
9  # | delta | 0.9.0 | 450 |
10 # | gamma | 1.5.1 | 450 |
11 # | alpha | 1.2.0 | 120 |

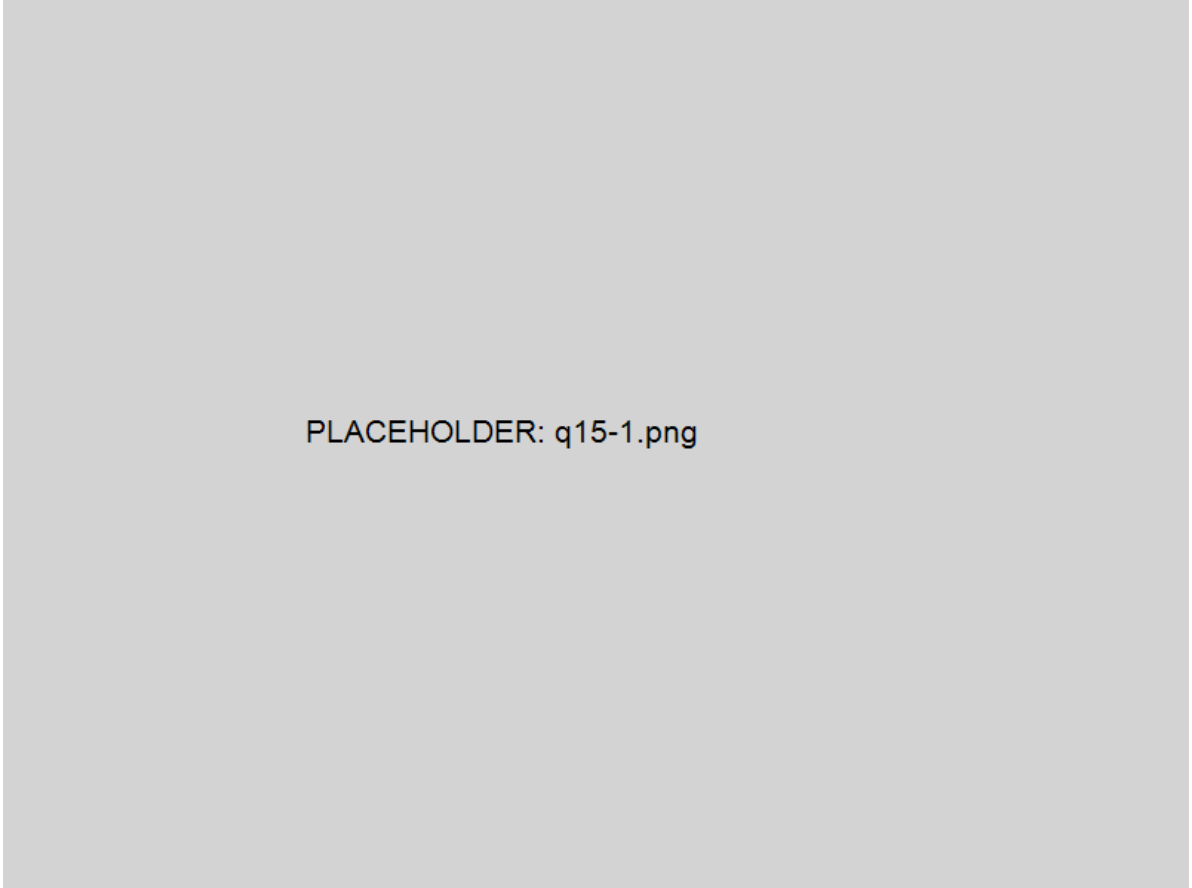
```

5.3 运行结果与截图

图 3 展示了 HTTP 服务启动、jq 安装与 api_report.sh 输出。

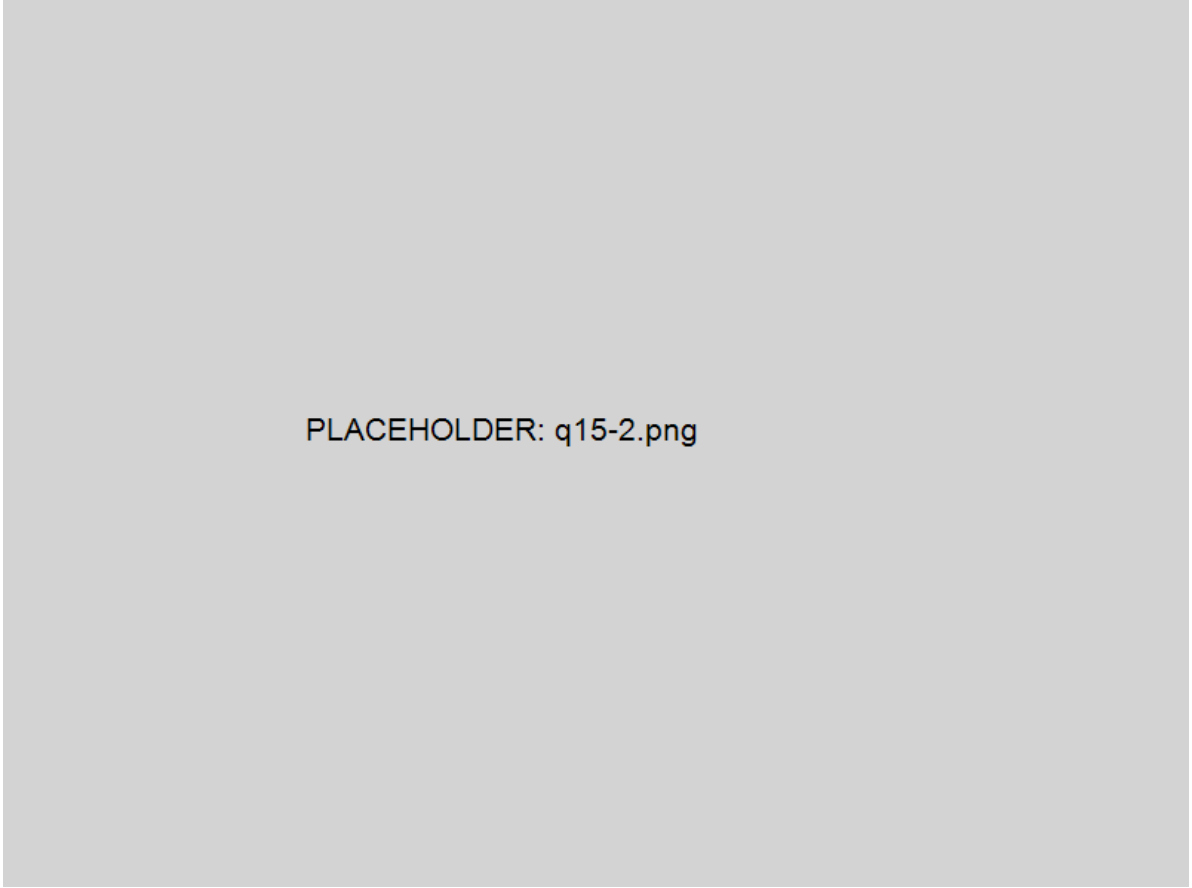
5.4 结果分析

- **curl -fsS:** -f 失败时静默退出码非零（不输出 HTML 错误页）；-s 静默模式不显示进度；-S 但仍在失败时输出错误信息；三者组合适合脚本中调用——成功无输出、失败有诊断；
- **jq 管道:** 第一个 jq 用 -s (slurp) 把数组包成单元素，再用 select(.status == "active" and .downloads >= 100) 筛选、sort_by(-.downloads, .name) 排序（负号 = 降序，字段名 = 升序）；第二个 jq 用 -r (raw output) 去掉 JSON 引号，用字符串模板直接拼 Markdown 表格；



PLACEHOLDER: q15-1.png

(a) `python3 -m http.server 8000` 后台启动、`sudo apt install -y jq`



PLACEHOLDER: q15-2.png

(b) `./api_report.sh` 执行 + `cat summary.md`: 3 个 active 且 `downloads` ≥ 100 的包，按 `downloads` 降序、`name` 升序排列

- 筛选结果验证: 5个包中, `beta` (inactive)、`epsilon` (downloads=80) 被排除, 剩下 `delta` (450)、`gamma` (450)、`alpha` (120), `delta` 和 `gamma` 同 `downloads` 时按 `name` 升序 (`delta` < `gamma`), 正确;
- `jq` 安装: WSL Ubuntu 24.04 默认未装 `jq`, 需 `sudo apt install -y jq`, 注意 `-y` 跳过交互确认。

第 6 章 q16: 修复并交付一个陌生的小型工具仓库

6.1 题目要求

主题: 课堂综合测试。复用第 13 题的项目, 在 q16 中完成一次 15 分钟综合检查:

- 复制 q13 为 q16 并初始化/继续使用 Git;
- 把问候语实现临时改成始终返回字面量 `Hello, name!`, 运行 `check.sh` 确认测试失败; 随后定位并修复;
- 编写最小 Makefile, 包含 `check`、`build` 和 `clean`; `check` 调用 `check.sh`, `build` 生成 `wheel`;
- 运行 `make check` 和 `make build`, 计算 `wheel` 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

6.2 实验步骤

复制 q13 为 q16, 初始化 Git:

```
1 cp -r q13 q16 && cd q16 && git init
```

临时改坏 `cli.py`:

```
1 # src/greetlab/cli.py (临时改坏)
2 def main():
3     p = argparse.ArgumentParser()
4     p.add_argument("--name", required=True)
5     a = p.parse_args()
6     if a.name.isspace():
7         sys.exit(2)
8     print("Hello, name!") # ← 改坏: 字面量而非 f-string
```

`./check.sh` 因 `pytest` 失败 (或 `ruff format check` 因引号/换行差异) 返回非零。修复后 Makefile:

```
1 .PHONY: check build clean
2
3 check:
4     bash check.sh
```

```

5
6 build:
7     python -m build
8
9 clean:
10    rm -rf dist build *.egg-info

```

最终验证与 SHA-256:

```

1 make check          # 通过
2 make build          # 生成 dist/greetlab_25020007117-0.1.0-py3-none-any.whl
3 sha256sum dist/*.whl # c77e45a73e704f0aa98d2c516a334c35474c96fcf3f1b57dfad827cd6a3
4 git add -A && git commit -m "q16 修复并交付"

```

6.3 运行结果与截图

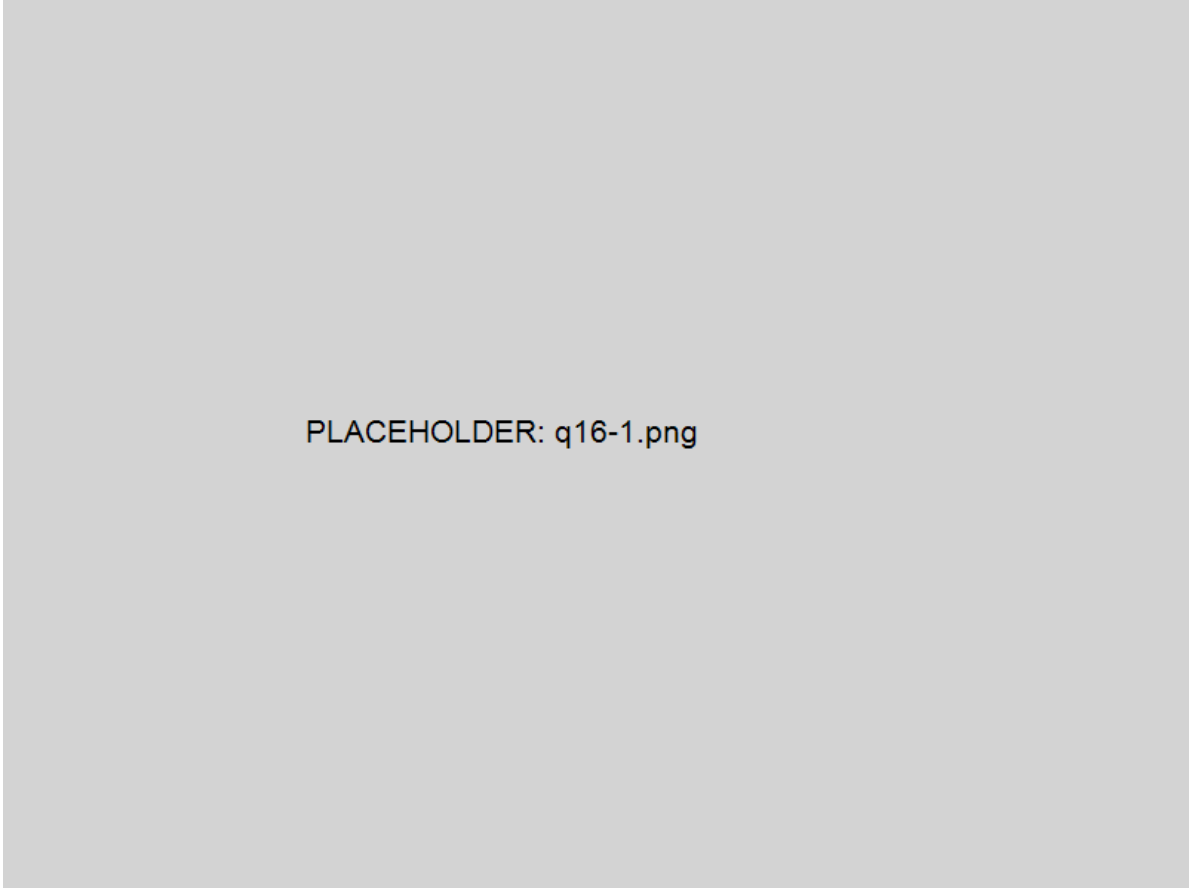
图 4 展示了从临时改坏、check.sh 失败、修复、make check/build 到 SHA-256 计算的完整综合测试。

6.4 结果分析

- **临时改坏与定位:** 把 `print(f"Hello, {a.name}!")` 改成字面量 `print("Hello, name!")` 后, `test_normal_name` 会因输出不匹配失败; 实际流程中先遇到 `ruff format --check` 报 `unformatted: File would be reformatted` (因为引号风格不一致), 用 `ruff format src/greetlab/cli.py` 修复格式问题后, 再通过 `pytest` 定位行为缺陷, 两层防护各司其职;
- **Makefile 简化:** `check` 仅调用 `bash check.sh`, 不重复写三条命令——单一来源原则; `build` 直接跑 `python -m build`; `clean` 清理 `dist`、`build` 和 `*.egg-info`, 比 `q14` 的 `rm -f` 更彻底 (`-rf` 递归删除目录);
- **SHA-256:** `sha256sum dist/greetlab_25020007117-0.1.0-py3-none-any.whl` 输出 `c77e45a73e704f0` 用于 wheel 分发后的完整性校验;
- **综合测试反思:** 15分钟时限下, 流程是”初始化 Git → 改坏确认失败 → 修复 → 质量门禁 → 构建 → 计算哈希 → 单一提交”; 实际截图显示 `make build` 首次因 `venv` 未激活 (`python -m build` 找不到模块) 失败, 激活后重试通过, 提示 Makefile 中的 `python` 依赖于当前 shell 的 `PATH`, CI 环境中应显式指定 `venv` 路径。

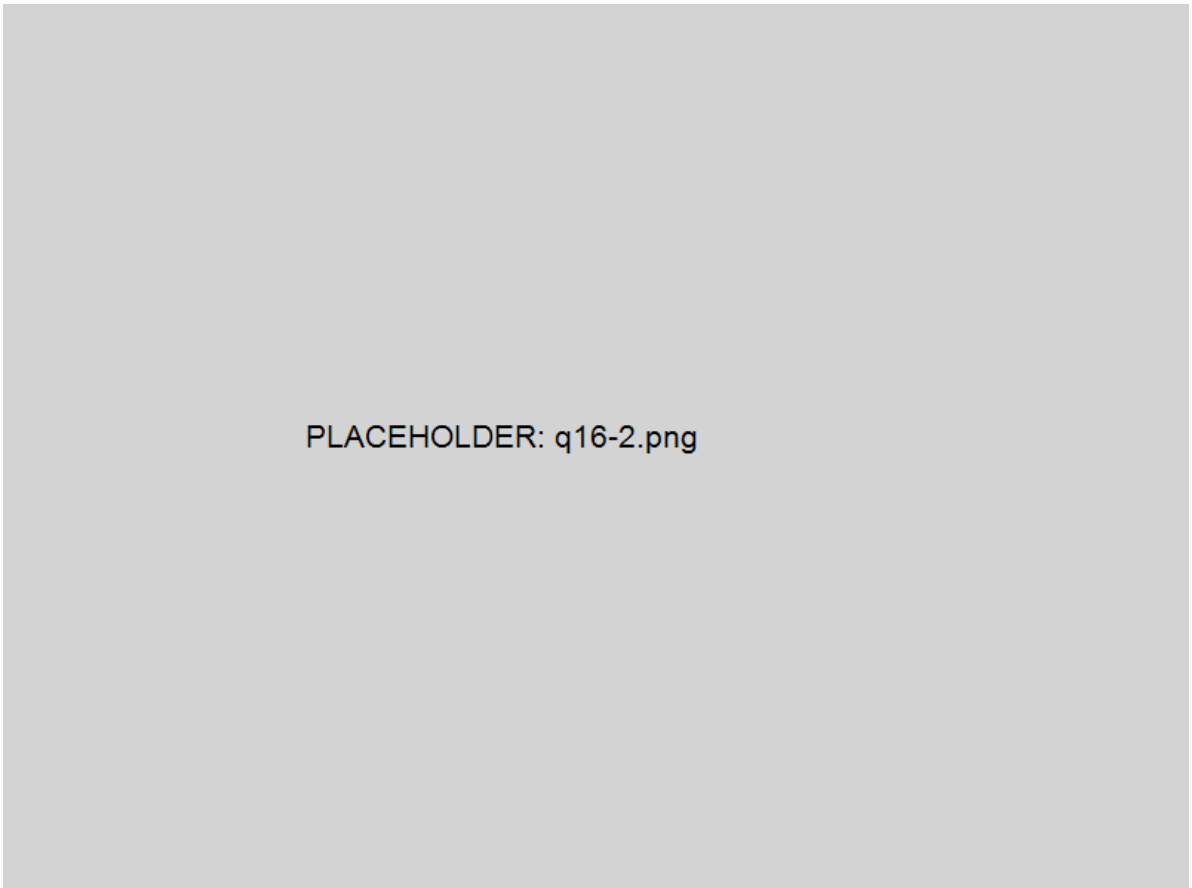
第 7 章 代码与仓库

项目已托管至 GitHub: <https://github.com/liangbaikai-666/tool-25020007117>



PLACEHOLDER: q16-1.png

(a) `cp -r q13 q16`、临时改坏 `cli.py`、`./check.sh` 因 `ruff format check` 失败、`ruff format` 修复后通过



PLACEHOLDER: q16-2.png

(b) `make check` 通过、`make build`¹³ 生成 wheel、`sha256sum dist/*.whl`

图 4: q16: 综合测试——改坏修复、Makefile 与 SHA-256

7.1 Git 提交记录

git log --all --graph --decorate --oneline 输出如下:

```
* 5f06c76 更新git log并重新编译report3
* 4d10f5e report3.pdf 编译产物
* dab5537 q09-q12 实验截图
* bd3537f report3.tex 实验报告
* 1d4d1d3 q12 PyTorch线性回归训练
* c804adb q11 协作材料改写
* 4a05b9d q10 清理嵌套目录
* 6a84240 q10 AI驱动修复循环
* d6a240d q09 包结构与源码
* 2b65ccd 第四次作业: q16 重新复制q13并添加Makefile
* 09f66c7 第四次作业: q13-q16 题目文件预创建
* 296f8e3 第四次作业: q13 check.sh 格式对齐实际输出
* 0a9ea09 第四次作业: q15 api_report.sh 对齐实际输出格式
```

其中 09f66c7 与 296f8e3 为第四次作业的主要提交, 覆盖 q13-q16 的源码、Makefile、shell 脚本和测试。

第 8 章 结论

通过本次实验, 完成了四道课上实验题, 主要收获如下:

- **可执行质量门禁 (q13):** 建立了 `ruff format --check → ruff check → pytest` 三层串联的 `check.sh`, `set -euo pipefail` 保证任何环节失败即退出; `pyproject.toml` 统一配置 `ruff` 和 `pytest`, `pip install -e .` 解决 `src-layout` 包的 `ModuleNotFoundError`;
- **Make 增量构建 (q14):** 正确声明 `all → report.txt → stats.txt → data.csv` 依赖链, `.PHONY` 声明 `all` 和 `clean`; 二次 `make` 无改动时 `Nothing to be done`, `touch data.csv` 后联动重建, `make clean` 只删生成物;
- **API + jq + Markdown (q15):** `python -m http.server 8000` 快速搭建本地 API, `curl -fsS` 适合脚本调用, `jq` 管道用 `-s slurp + select 筛选 + sort_by(-.downloads, .name)` 排序 + `-r raw output` 生成 Markdown 表格;
- **综合测试 (q16):** 15 分钟内完成”改坏 → 确认失败 → 修复 → 质量门禁 → 构建 → SHA-256 → Git 提交”; Makefile 三目标 (`check / build / clean`), wheel 哈希 `c77e45a7...cd6a3` 用于完整性校验。

总体而言, 本实验覆盖了”质量门禁 → 构建系统 → 数据处理 → 综合交付”四个开发工具主题: 从让 `check.sh` 成为每次提交前必须跑的门槛, 到用 Makefile 精确管理文件依赖避免无效重建, 再到用 `curl + jq` 把原始 JSON 转成工程师可读的 Markdown 报告, 最后在 15 分钟综合测试中走完”改坏修复 → 构建交付”的完整链路。全部代码、截图与报告已提交至 GitHub (`origin/main`), 便于回溯与复查。